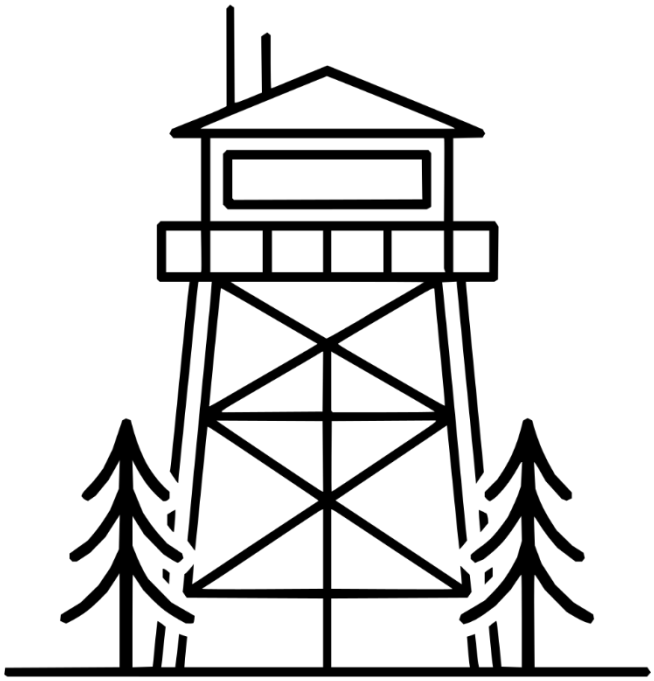


Watchtower EDR

BY: **Brenden Ferguson**

A cyber-defense solution for everyone.

Table of Contents

Executive Summary 3
Project Definition Document 4
Technical Requirements Document 6
1. Introduction 6
2. General Description 8
3. Specific Requirements 9
4. Appendices 12
5. Glossary 16
6. References 18
Project Schedule 19
Status Reports 21
Status Report 1 21
Status Report 2 22
Status Report 3 23
Status Report 4 24
Status Report 5 25
Status Report 6 26
User's Guide 27
1. Server Section 27
1.1 Installation 27
1.2 Configuration 28
1.3 The Dashboard 29
1.4 Agents Page 30
1.5 Software Inventory Page 31
1.6 Vulnerability Management Page 32
1.7 Reporting Page 32
1.8 Settings Page 33
2. Agent Section 34

2.1 Windows Installation 34
2.2 Linux Installation.....35

Executive Summary

Opportunity

In the modern landscape, information security is critical, yet it is often the first function sacrificed due to limited resources. Small-to-medium-sized IT departments frequently face "information overload" and a lack of specialized cybersecurity expertise. This makes managing endpoints difficult in a rapidly evolving threat landscape. Traditional Endpoint Detection and Response (EDR) platforms are often cost-prohibitive and complex to configure, leaving smaller organizations with significant gaps in their digital defenses.

Mission

Watchtower EDR provides resource-limited IT professionals with a high-visibility “eye in the sky” for their digital infrastructure. The project’s goal is to strip away the complexity of traditional security tools, offering a system that is simple to deploy yet provides deep, insightful data. By bridging the gap between raw hardware telemetry and actionable intelligence, Watchtower EDR enables IT teams to transition from reactive “firefighting” to a proactive, risk-based defense strategy.

Solution

Watchtower EDR is a unified environment comprising lightweight, cross-platform agents and a self-hosted central management server. The system automates vulnerability tracking through a streamlined three-stage pipeline:

- **Collection:** Agents perform non-intrusive auditing of installed software and system information.
- **Normalization:** A custom engine maps “messy” software data to **NIST Common Platform Enumeration (CPE)** URIs.
- **Correlation:** The server cross-references this inventory against the **National Vulnerability Database (NVD)** to identify threats in real-time.

Advantages

Unlike enterprise alternatives that require complex integration, Watchtower EDR is a self-contained, “plug-and-play” solution. While many EDRs focus solely on active threats, Watchtower prioritizes **“Vulnerability Awareness,”** identifying the underlying risks within an organization's software inventory before they can be exploited.

Project Definition Document

Ask yourself:	Also known as:
Why am I doing this?	Purpose
Small and medium-sized organizations often struggle to meet cybersecurity goals and standards due to limited manpower and expertise within their IT departments. Even basic tasks—such as keeping up with current events and vulnerabilities, maintaining application installations, and ensuring systems remain up to date—can become challenging. I work in an environment such as this, where no one is consistently available to manage the organization's security operations. This solution is designed to make achieving those security goals more practical and attainable.	
What goal(s) does this project support, what problems are solved, what needs are met?	Goals & Objectives
• This solution will enable organizations to stay informed about installed software and any known vulnerabilities associated with those versions. It will also provide insight into additional common attack surfaces, such as file integrity. • The solution addresses the challenge many IT departments face in managing potential threats through known and preventable means, often due to limited time, lack of expertise, or the complexity of existing tools. • This product empowers IT departments to remain aware of developments in the cybersecurity landscape and maintain greater control over the systems deployed within their organizations.	
How does this project fit with other solutions that available?	Scope, Project Context, Project Dependencies
This project does not aim to compete directly with the robust capabilities of industry-standard Extended Detection and Response (XDR) solutions. Instead, it focuses on delivering essential functionality without the complexity typically associated with those platforms. The goal is to provide organizations with an effective, easy-to-deploy solution that balances functionality and simplicity. Additionally, this product stands out for its ability to generate intelligence by comparing installed software against known vulnerabilities found in Common Vulnerabilities and Exposures (CVE) databases.	
What is the expected benefit from this project?	Expected Benefits, Business Case, Value, Success Criteria

This solution is designed to help small and medium-sized IT departments achieve their cybersecurity goals by providing insight into potential attack surfaces through analysis of installed software and known vulnerabilities. By identifying these risks, the solution helps mitigate threats and reduce the likelihood of financial or data loss resulting from a cyberattack. Additionally, this Endpoint Detection and Response (EDR) solution is intentionally simpler to deploy than many existing alternatives, enabling organizations without dedicated cybersecurity professionals to enhance their overall security posture.

What am I going to do?	Scope

- I am going to build a backend server in Python to handle API calls and data parsing
- I am going to build deployable agents in Go that will handle gathering data, such as installed software, on each client and relay it to the server
- I am going to implement Rest API for secure client-server communication
- I am going to construct a web environment for managing the server and viewing information
- I am going to perform extensive research and test at every stage to ensure proper functionality

Who is affected by this and who must be involved?	Stakeholders

1. Small/Medium IT Departments
2. Organization Management
3. Cybersecurity professionals

How will we know when I am done and whether the project was successful?	Success Criteria

The project will be considered successful when the server environment can be deployed and managed with relative ease, agents can be successfully deployed to collect relevant information from client systems, and the solution is reviewed and approved by a cybersecurity professional.

Completed by:	Brenden Ferguson
Date:	1/20/2026
Project Title / ID Number	Watchtower EDR
Approved by:	Dr. Cheri Kembell

Technical Requirements Document

Watchtower EDR: Technical Requirements Document (version 3.0)

Project Name:	**Watchtower EDR**
Date:	April 21, 2026
Prepared by:	Brenden Ferguson
Document status (draft, proposed, validated or approved)	Approved

1. Introduction

This document contains the system requirements for **Watchtower EDR**. These requirements have been derived from several sources, including industry experience, SQLite documentation, the **National Institute of Standards and Technology (NIST)**, and Go development documentation.

1.1 Purpose of This Document

This document is intended to guide development of **Watchtower EDR**. It will go through several stages during the course of the project:

1. **Draft:** The first version, or draft version, is compiled after requirements have been discovered, recorded, classified, and prioritized.
2. **Proposed:** The draft document is then proposed as a potential requirements specification for the project. The proposed document should be reviewed by several parties, who may comment on any requirements and any priorities, either to agree, to disagree, or to identify missing requirements. Readers include end-users, developers, project managers, and any other stakeholders. The document may be amended and re-proposed several times before moving to the next stage.
3. **Validated:** Once the various stakeholders have agreed to the requirements in the document, it is considered validated.
4. **Approved:** The validated document is accepted by representatives of each party of stakeholders as an appropriate statement of requirements for the project. The developers then use the requirements document as a guide to implementation and to check the progress of the project as it develops.

1.2 How to Use This Document

We expect that this document will be used by people with different skill sets. This section explains which parts of this document should be reviewed by various types of readers.

Types of Reader

- **Developers (Go, Backend/Frontend):** Focus on Sections 3 (Specific Requirements) and 4 (Appendices).
- **IT Departments/End-Users:** Focus on Sections 1.3 (Scope), 2 (General Description), and 3.1 (User Requirements).

- **Project Managers:** Review Sections 1.4 (Business Case), 2.4 (Constraints), and 7 (Index/References).
- **Cybersecurity Professionals:** Focus on Sections 1.3 (Scope), 3 (Specific Requirements), and 4 (Appendices).

Technical Background Required

- **IT Networking:** Basic understanding of Client-Server architecture and APIs.
- **Cybersecurity Basics:** Familiarity with common terms like Vulnerabilities, Exploits, and the role of an EDR.
- **Go Environment:** For developers, a working knowledge of the Go programming language and SQLite database management is required

Overview Sections

- 1.3 (Scope)
- 1.4 (Business Case)
- 2 (General Description)

Reader-Specific Sections

- **Developers & Cybersecurity Professionals:** Should prioritize **Section 3.2 (System Requirements)** and **Section 4 (Appendices)** for technical implementation details, such as database schemas and data flow logic.
- **IT Departments & Project Managers:** May skip **Section 4 (Appendices)** as it contains low-level technical diagrams (e.g., SQL ERDs and JSON response structures) not required for high-level oversight.

Section Order Dependencies

No section order dependencies.

1.3 Scope of the Product

Watchtower EDR is a lightweight and simplistic **Endpoint Detection and Response (EDR)** solution designed for small/medium-sized organizations with limited IT/cybersecurity resources. It aims to provide:

- Software inventory tracking (installed applications and versions)
- Vulnerability analysis (comparing installed software to **Common Vulnerabilities and Exposures (CVE)** databases).
- **File integrity monitoring (FIM)** (basic attack surface analysis).
- Simplified deployment (easy-to-use agents and server).

Key limitations include:

- Agent-based deployment: Requires agent deployment on client systems
- No real-time threat response: Alerts and reports are provided for manual intervention
- Not a full **Extended Detection and Response (XDR)** or antivirus replacement: Focuses on core EDR functionality, without advanced threat hunting, network monitoring, or virus/malware detection.

1.4 Business Case for the Product

Problem:

Small and medium-sized organizations often struggle to meet cybersecurity goals and standards due to limited manpower and expertise within their IT departments. Even basic tasks—such as keeping up with current events and vulnerabilities, maintaining application installations, and ensuring systems remain up to date—can become challenging. Current solutions, such as XDR systems, are often considered too daunting for non-experts to deploy and manage.

Solution:

Watchtower EDR provides:

- Automated vulnerability scanning with CVE integration
- Simplified deployment to empower non-experts to improve security posture
- Actionable insights to aid IT teams in risk mitigation

1.5 Overview of the Requirements Document

1. **Straightforward Installation & Configuration:** Both server and agents can be installed and configured with both minimal steps and deep technical knowledge.
2. **Data Safety and Control:** Data and transactions must be secured via authentication, encryption, and local storage to keep organizations in control of their own data.
3. **Automated Data Lifecycle:** Agent data, vulnerabilities, and NIST data update automatically on set schedules to make system management simpler for the end-user.

2. General Description

This section will give the reader an overview of the project, including why it was conceived, what it will do when complete, and the types of people we expect will use it. We also list constraints that were faced during development and assumptions we made about how we would proceed.

2.1 Product Perspective

Watchtower EDR addresses the gap between limited IT resources and cybersecurity needs in small/medium-sized organizations. It is developed by **Brenden Ferguson** and targets:

- **Stakeholders:** IT departments, organization management, cybersecurity professionals
- **Users:** IT staff with basic technical skills

2.2 Product Functions

- **Software Inventory**: Aggregates comprehensive lists of installed applications and versions from all registered endpoints into a centralized database.
- **Vulnerability Mapping**: Automatically maps local software inventory to standardized **CPE URIs** to identify active **CVE** threats.
- **File Integrity Monitoring**: Detect and report modifications to critical system files to provide alerts of potential shifts on an attack surface.
- **Alerting**: Generate visual alerts categorized by **CVSS severity scores** to aid in risk-based response.

2.3 User Characteristics

- **IT Staff**
  - **Technical Background:** Basic IT knowledge.
  - **Motivation:** Improve organizational security posture with limited resources and cybersecurity knowledge.
  - **Obstacles:** Time constraints, lack of technical expertise
- **Cybersecurity Professionals**
  - **Technical Background:** Advanced knowledge of vulnerability/attack-surface management and compliance standards.
  - **Role:** Testing and validation of application functionality in the development cycle.

2.4 General Constraints

- **Platform Compatibility:** All server components and agent binaries must be fully compatible with **Microsoft Windows 11**.
- **Development Stack:** The backend logic and agent services are built using **Go (Golang)**, with local data handled by **SQLite** and high-speed string normalization managed by the **Bluge** indexing engine. The server installation is handled by Inno Compiler.
- **Frontend UI:** A responsive web dashboard powered by **Go HTML Templates**, **Vanilla JavaScript**, **JSON**, and **CSS3**.
- **Security Architecture:** All communication is secured via **TLS 1.3** and **AES-256-GCM** encryption, utilizing a private **X.509 Self-Signed Certificate** infrastructure and **JSON Web Tokens (JWT**) for session authentication.
- **Dependencies: NIST NVD API**.

2.5 Assumptions and Dependencies

- **Assumptions**
  - Organizations have existing IT infrastructure.
  - Organizations possess administrative permission to deploy agents on endpoints.
- **Dependencies**
  - Access to NIST NVD API for **CPE** and **CVE** data source.
  - Windows machine to deploy server environment on.
  - Secure network for client-server communication.
  - Reliable **DNS resolution** to the server host to ensure reliable agent-server connection.

3. Specific Requirements

This section of the document lists specific requirements for **Watchtower EDR**. Requirements are divided into the following sections:

1. **User requirement:** These are requirements written from the point of view of end users, usually expressed in narrative form.
2. **System requirements:** These are detailed specifications describing the functions the system must be capable of doing.
3. **Interface requirements:** These are requirements about the user interface, which may be expressed as a list, as a narrative, or as images of screen mock-ups.

3.1 User Requirements

- **Software Visibility**: Users can view detailed client information, specifically focusing on installed software names and versions, via a centralized web console.
- **Low-Barrier Deployment**: IT staff can deploy the server environment and endpoint agents with minimal technical effort, catering to organizations with limited manpower.
- **Proactive Alerting**: Users receive visual alerts on the web console immediately when known vulnerabilities (CVEs) or file integrity threats are discovered.
- **Compliance Reporting**: Users can generate on-demand reports for auditing or security compliance to aid in risk mitigation.
- **Integrity Monitoring**: Users can monitor unauthorized file changes on client systems to detect potential attack surface shifts.

3.2 System Requirements

3.2.1 Backend Server

- **Client Telemetry Storage**: The server must parse and store registered client attributes, including hostname, IP address, connection timestamps, and comprehensive software inventories.
- **NVD Synchronization Engine**:
  - The server must implement a 12-hour synchronization cycle with the NIST NVD API.
  - To comply with NIST constraints, the sync logic must utilize a 120-day sliding window for date-range queries.
  - The engine must support pagination (5,000 records per page) using NIST's startIndex and resultsPerPage parameters.
- **Vulnerability Mapping**: The system must compare installed software against the locally mirrored CPE/CVE data to identify risks without constant external API calls.
- **Database Management**:
  - All configuration, telemetry, and NVD data must be stored in a local SQLite environment.
  - The database must be configured with **Write-Ahead Logging (WAL)** and a **Busy Timeout** to handle concurrent writes from multiple agents without locking failures.
- **Graceful Lifecycle**: The server must capture OS signals (SIGTERM/Interrupt) to trigger a graceful shutdown, ensuring all background workers stop and database connections close cleanly.
- **Secure Communication**: Provide HTTPS-secured REST API endpoints for passing data to the web UI, and action-based API endpoints for agent interactions. It must accomplish this by using an automatically managed **self-signed certificate**.

3.2.2 Agents

- **Data Collection & Heartbeat**: Agents must collect device attributes and software lists, communicating them securely to the backend at a user-defined frequency.
- **Background Operation**: Agents must run as a persistent background service with little-to-no user interaction after the initial installation.

- **Intermittent Probing**: Agents must "heartbeat" to the server to check for configuration updates or pending telemetry requests.
- **Automated Installer**: The installer package must build the service, establish the initial server handshake, and register the service with the Windows OS.

3.2.3 Web Dashboard

- **Intuitive Visualization**: Provide a web-based UI to view system status, manage client lists, and review software deployments and vulnerability reports.
- **Centralized Configuration**: Allow users to adjust global settings, such as agent reporting frequencies and the NVD API Key, directly from the interface.

3.3 Interface Requirements

- **Navigation & Layout**: The dashboard must feature a sidebar-driven menu for intuitive navigation between the Dashboard, Agents, Software Inventory, Vulnerability Management, Reporting, and Settings pages.
- **Alert Presentation**: High-priority vulnerabilities must provide clear, visual cues to indicate severity.
- **Dynamic Reporting**: The Software and Vulnerability pages must include client-side search bars and filtering.
- **Authentication**: The web interface must utilize **JWT** for secure user session management and protected routing.

4. Appendices

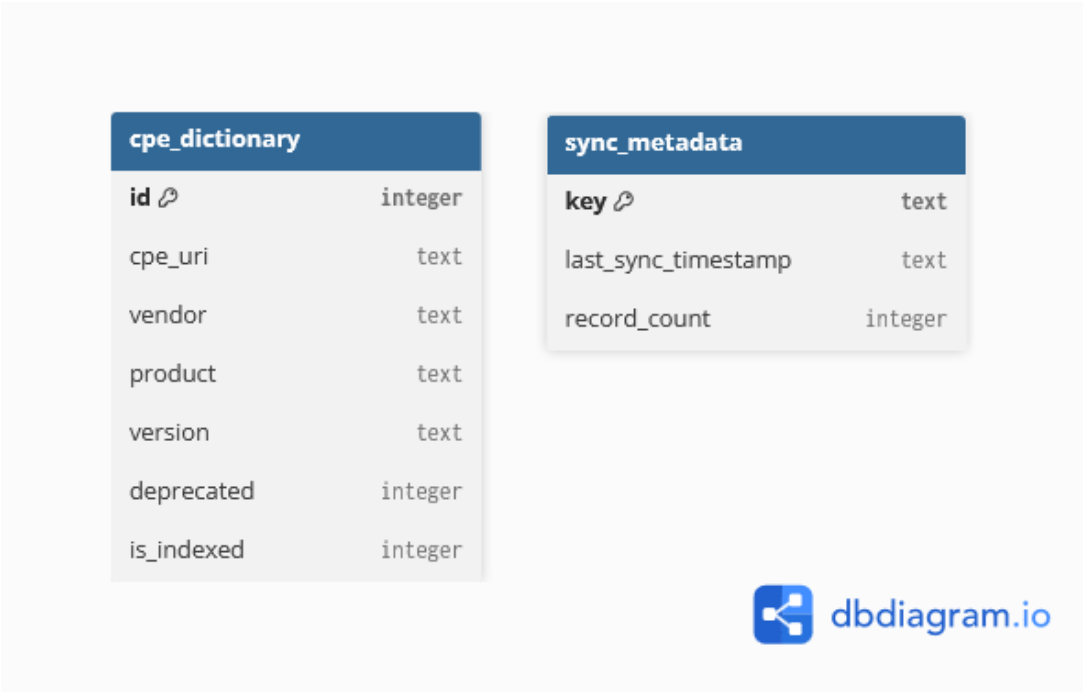
4.1 Database Schemas

The system utilizes a **multi-file database architecture**, segregating data into discrete SQLite instances based on functional roles (e.g., Global Vulnerability Data vs. Local Agent Telemetry). This modular approach minimizes **database-level lock contention** during high-volume concurrent operations, such as NIST CPE synchronization. Furthermore, this isolation establishes a **fault-containment boundary**, ensuring that localized file corruption does not compromise the integrity of the entire data ecosystem.

4.1.1 Main Database Schema

4.1.2 Users Database Schema

4.1.3 CPE Database Schema

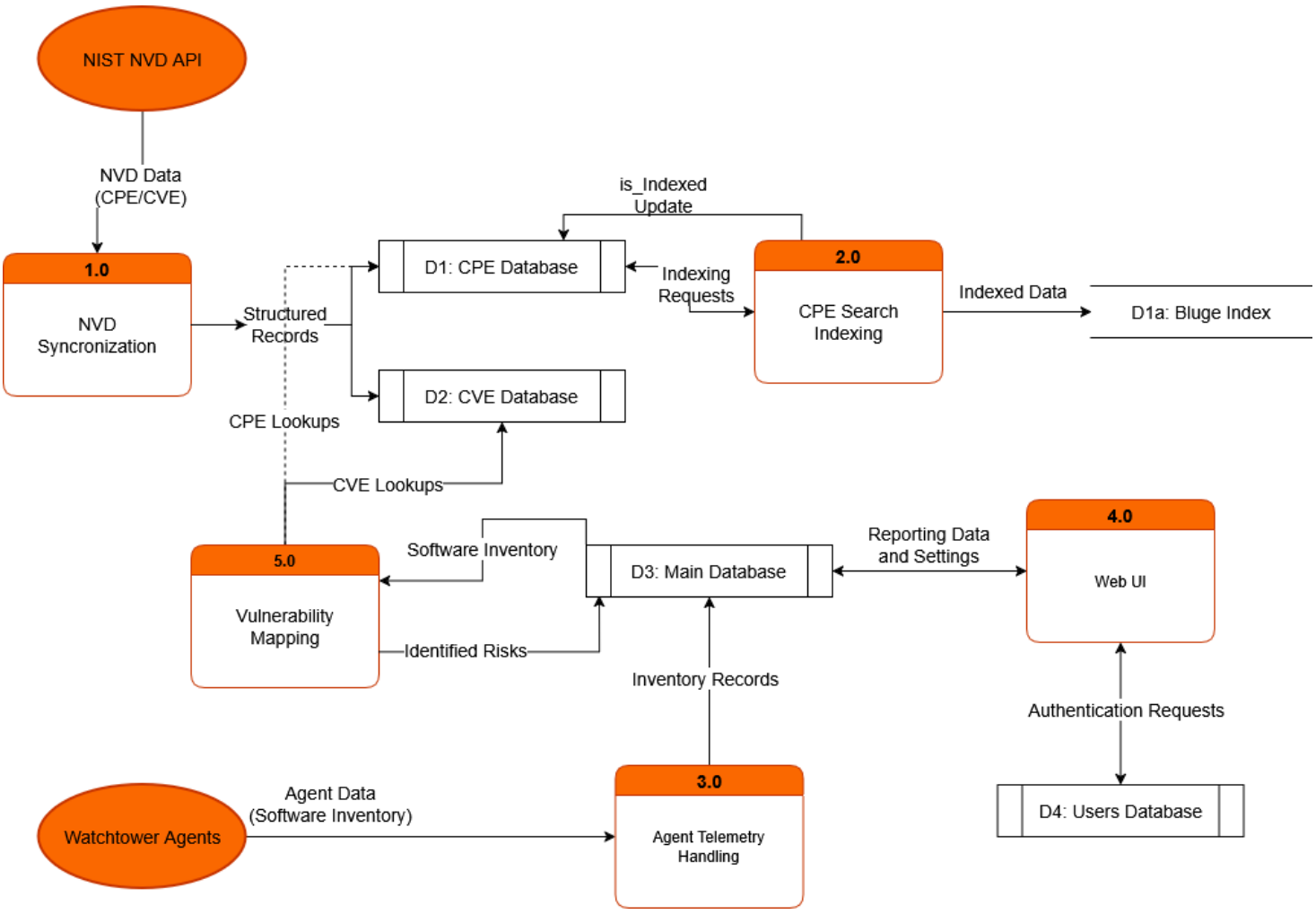


4.1.4 CVE Database Schema

Note on Cross-Database Relationship: *cpe_uri serves as a common identifier between cpe_dictionary (CPE Database) and software_vulnerabilities (CVE_Database), but this relationship is maintained solely at the* ***Application Layer*** *and is not shown above.*

4.2 Data Flow Diagram

Level 1 DFD



5. Glossary

General Cybersecurity & Industry Terms

- **CVE (Common Vulnerabilities and Exposures)**: A list of publicly disclosed computer security flaws.
- **EDR (Endpoint Detection and Response)**: A solution that monitors endpoint devices to detect and respond to cyber threats.
- **Exploit**: A technique used to breach the security of a system in violation of security policy.
- **FIM (File Integrity Monitoring)**: A security process that monitors and alerts on unauthorized changes to operating system, application, and configuration files.
- **NIST (National Institute of Standards and Technology)**: The U.S. federal agency that develops technology, metrics, and standards.
- **NVD (National Vulnerability Database)**: The U.S. government repository of standards-based vulnerability management data.
- **Vulnerability**: A specific weakness that renders an asset open to exploitation by a threat.
- **XDR (Extended Detection and Response)**: An evolution of EDR that integrates security data across multiple layers (network, cloud, etc.).

Networking & API Integration

- **AES-256-GCM:** A high-speed authenticated encryption algorithm. It uses a 256-bit key for Advanced Encryption Standard (AES) in **Galois/Counter Mode (GCM)** to provide both confidentiality and data integrity.
- **API (Application Programming Interface)**: A set of protocols and tools for building software applications.
- **API Key**: A unique code used to identify and authenticate a specific application or user.
- **CSS3 (Cascading Style Sheets Level 3)**: The latest standard of the language used for describing the presentation and styling of web pages.
- **GCM (Galois/Counter Mode)**: A mode of operation for symmetric-key cryptographic block ciphers that provides both data authenticity and confidentiality with high performance.
- **HTTP (HyperText Transfer Protocol)**: The foundational protocol used for transferring data over the web.
- **HTTPS (HyperText Transfer Protocol Secure)**: An encrypted version of HTTP that uses TLS to secure data in transit.
- **JavaScript**: A high-level, interpreted programming language primarily used to create interactive and dynamic content on web browsers.
- **JSON (JavaScript Object Notation)**: A lightweight, text-based data-interchange format that is easy for humans and machines to read and write.
- **JWT (JSON Web Token)**: A compact, URL-safe means of representing claims to be transferred between two parties. It is commonly used for authentication and session handling.
- **REST API (Representational State Transfer)**: An architectural style for providing standards between computer systems on the web.
- **TLS (Transport Layer Security) & TLS 1.3**: A cryptographic protocol designed to provide communications security over a computer network. **TLS 1.3** is the latest version, offering improved performance and security by removing legacy algorithms and shortening the handshake.

- **Vanilla JavaScript**: Refers to using the **JavaScript** language as-is without any external libraries or frameworks.
- **Self-Signed Certificate (X.509):** A digital certificate that is signed by the same entity whose identity it certifies, rather than a trusted Certificate Authority (CA). Often used in testing or internal environments.

Database & Backend Performance

- **Bluge:** A high-performance, open-source indexing and search library written in Go, designed to provide advanced full-text search capabilities.
- **Busy Timeout**: A configuration for SQLite that instructs the database to wait for a specified duration when a table is locked, rather than returning an immediate error.
- **SQL (Structured Query Language)**: the standard programming language used specifically for managing, querying, and manipulating data held in relational databases.
- **SQLite**: A C-language library that implements a small, fast self-contained, high-reliability, full-featured, **SQL** database engine.
- **WAL (Write-Ahead Logging)**: A high-concurrency logging method for SQLite that allows multiple readers to access the database simultaneously while a write operation is in progress.

6. References

1. Automox, “Bad cyber hygiene: 60% of breaches tied to unpatched vulnerabilities,” *Automox Blog*. [Online]. Available: https://www.automox.com/blog/bad-cyber-hygiene-breaches-tied-to-unpatched-vulnerabilities
2. Bluge Labs, “Bluge Labs.” [Online]. Available: https://blugelabs.com/
3. CISA, “NICCS cybersecurity glossary,” *National Initiative for Cybersecurity Careers and Studies*. [Online]. Available: https://niccs.cisa.gov/resources/glossary
4. E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” *Internet Engineering Task Force (IETF)*, RFC 8446, Aug. 2018. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446
5. Fortinet, Cybersecurity glossary,” *Fortinet*. [Online]. Available: https://www.fortinet.com/resources/cyberglossary/
6. Go, “Documentation,” *The Go Programming Language*. [Online]. Available: https://go.dev/doc/
7. Help Net Security, “Cybersecurity attack attempts increased by 30% in 2023,” *Help Net Security*, Jan. 25, 2024. [Online]. Available: https://www.helpnetsecurity.com/2024/01/25/cybersecurity-attack-attempts-increase/
8. JRSoftware, "Inno Setup Help," jrsoftware.org. [Online]. Available: https://jrsoftware.org/ishelp/index.php. [Accessed: Apr. 30, 2026].
9. NIST, “National Vulnerability Database,” *National Institute of Standards and Technology*. [Online]. Available: https://nvd.nist.gov/
10. Sophos, “The cybersecurity skills shortage in smaller businesses,” Sophos, White Paper. [Online]. Available: https://assets.sophos.com/X24WTUEQ/at/gqpx2zb7wpnzcgpz7xp4vmnr/sophos-the-cybersecurity-skills-shortage-in-smaller-businesses-wp.pdf
11. SQLite, “Write-Ahead Logging,” *SQLite*. [Online]. Available: https://www.sqlite.org/wal.html
12. TechTerms, “TechTerms: The Computer Dictionary.” [Online]. Available: https://techterms.com/

Project Schedule

Jan 2026
15
Feb
Mar
Apr
May
30 Stand-Up 8 BF
31 Presentation BF
32 Initialize GO module BF
33 Obtain API key for ... BF
34 Design SQLite Sche... BF
35 Create login page a... BF
36 Clone NIST CPE dat... BF
37 Create virtual machi... BF
38 Install Insomnia to t... BF
39 Status Meeting 1 BF
40 Status Meeting 6 BF
41 Design web pages BF
42 Develop HTML fram... BF
43 Implement Web UI ... BF
44 Create Agent Modu... BF
45 Create function to p... BF
46 Create function wit... BF
47 Create server respo... BF
48 Create listening API... BF
49 Create API POST to ... BF
50 Create functions to ... BF
51 Create function to s... BF
52 Build Report Queries BF
53 Propogate web ui r... BF
54 Build routes to upd... BF
55 Build function to ha... BF
56 Test Web UI and fin...
57 Build Installation pa... BF
58 Create logger functi... BF
59 Create GitHub Repo BF
60 Implement Configu... BF
61 Live config updatin... BF
62 Generate certificate...

Status Reports

Status Report 1

Project Status Report

● Overall Status: **On Track**

Project Name: Watchtower EDR

Date: 2/9/2026

Status Code Legend

● On Track: Project is on schedule

● High Risk: At risk, with a high risk of going off track

● At Risk: Milestones missed but date intact

● Off Track: Date will be missed if action not taken

The project is **On Track** the week of 2/9/26 – 2/20/26, due to the following:	● To add a bullet for any status, on the Home tab, in the Styles gallery, select the name of the status of you need. ● To replace any placeholder text (such as this) just tap it and start typing. ● Additional status item.
Issues:	● Getting started with coding portion of development
Milestones accomplished the week of 2/9/26 – 2/20/26:	
Milestones planned this week, but not achieved with variance:	● Setup Basic Web Server in Go to deliver default page ● Setup SQLite file ● Begin cloning NIST CVE
Milestones planned for next week:	● Setup virtual machines for testing ● Develop web login and security ● Further develop website interface for home page
Areas/questions for discussion:	

Tasks Complete: 7 Tasks Remaining: 23 Tasks Late: 2

Status Report 2

Project Status Report

Overall Status: **On Track**

Project Name: Watchtower EDR

Date: 2/24/2026

Status Code Legend

- On Track: Project is on schedule
- High Risk: At risk, with a high risk of going off track
- At Risk: Milestones missed but date intact
- Off Track: Date will be missed if action not taken

The project is **On Track** the week of 2/23/26 – 3/6/26, due to the following:	In the past two weeks, I have made major progress in getting data pulled down from NIST NVD, implementing web authentication, and constructing many of the building blocks my project will be relying on throughout its lifecycle.
Issues:	• Working out SQL schema • Contemplating Web layout
Milestones accomplished the week of 2/9/26 – 2/20/26:	• NIST Dictionary table successfully pulling data!!! • Web dashboard authentication functional • Useful logging implemented • Source control implemented
Milestones planned this week, but not achieved with variance:	• Design Web Pages • Finalize SQL Schema • Develop HTML frame and scripts
Milestones planned for next week:	• Initialize agent modules • Create basic connection to server • Begin creating function to collect agent data
Areas/questions for discussion:	

Tasks Complete: 20 Tasks Remaining: 41 Tasks Late: 2

Status Report 3

Project Status Report

Overall Status: **On Track**

Project Name: Watchtower EDR

Date: 3/9/2026

Status Code Legend

- On Track: Project is on schedule
- High Risk: At risk, with a high risk of going off track
- At Risk: Milestones missed but date intact
- Off Track: Date will be missed if action not taken

The project is **On Track** the week of 3/23/26 – 3/6/26, due to the following:	In the past two weeks, I have made major progress in getting data pulled down from NIST NVD, implementing web authentication, and constructing many of the building blocks my project will be relying on throughout its lifecycle.
Issues:	• Working out SQL schema • Contemplating Web layout
Milestones accomplished the week of 3/9/26 – 3/20/26:	• Agent modules created • Config.yaml successfully implemented • Self-healing SQL schema implemented • Custom ISO for virtual machines created • General web page design complete with basic routes
Milestones planned this week, but not achieved with variance:	• Create function to pull software data in agent • Create function to hash important files • Create API call and response for regular agent probe
Milestones planned for next week:	• Begin implementing data routes to web UI • Implement REST API data transactions between agent and server • Build basic SQL queries for agent data • Finalize agent data collection functions
Areas/questions for discussion:	

Tasks Complete: 28 Tasks Remaining: 34 Tasks Late: 2

Status Report 4

Project Status Report

Overall Status: **On Track**

Project Name: Watchtower EDR

Date: 3/23/2026

Status Code Legend

- On Track: Project is on schedule
- High Risk: At risk, with a high risk of going off track
- At Risk: Milestones missed but date intact
- Off Track: Date will be missed if action not taken

The project is **On Track** the week of 3/23/26 – 4/3/26, due to the following:	In the past two weeks, I have made major progress in constructing my agents, restructuring my project for a more unified format, furthering the web UI, and securing all communication.
Issues:	• Building the logo took such a long time
Milestones accomplished the week of 3/21/26 – 4/3/26:	• Agent module able to collect data • Server TLS communication secured • Server certificate and key dynamically generated • Web UI updated • Data pulling to dashboard • Settings page completely functional • Created functions and routes to send data to server from agent
Milestones planned this week, but not achieved with variance:	• Create routes and functions to receive data from agents • Create server functions to compare agent data to NIST data • Finish implementing “point-in-time” database rolling
Milestones planned for next week:	• Work on Agents and Reports pages on web UI once there is data in the backend • Continue working on server functions to receive and handle agent data and transactions
Areas/questions for discussion:	

Tasks Complete: 33 Tasks Remaining: 29 Tasks Late: 3

Status Report 5

Project Status Report

Overall Status: **At Risk**

Project Name: Watchtower EDR

Date: 4/6/2026

Status Code Legend

- On Track: Project is on schedule
- High Risk: At risk, with a high risk of going off track
- At Risk: Milestones missed but date intact
- Off Track: Date will be missed if action not taken

The project is **On Track** the week of 4/3/26 – 4/17/26, due to the following:	In the past two weeks, I have made progress with some of the documentation and getting server-side routes and handling developed for agent telemetry. I was not able to get the software telemetry handled due to time restrictions during Holy Week.
Issues:	Was not able to develop during Easter week due to business.
Milestones accomplished the week of 4/6/26 – 4/17/26:	Server able to receive and handle agent enrollment and heartbeat data Simulated agent enrollment and heartbeat signal Staged system for other types of telemetry and initialized routes Completed logo and TRD v2
Milestones planned this week, but not achieved with variance:	Handle agent software telemetry Begin working on matching software to vulnerabilities
Milestones planned for next week:	Begin building routes for pulling data to the web UI Build logic for live comparison of software to vulnerability listings Build calls to NIST for complete vulnerability data Get agent-side routes for telemetry established
Areas/questions for discussion:	

Tasks Complete: 44 Tasks Remaining: 18 Tasks Late: 4

Status Report 6

Project Status Report

Overall Status: **On Track**

Project Name: Watchtower EDR

Date: 4/30/2026

Status Code Legend

- On Track: Project is on schedule
- High Risk: At risk, with a high risk of going off track
- At Risk: Milestones missed but date intact
- Off Track: Date will be missed if action not taken

The project is **On Track** the week of 4/18/26 – 4/30/26, due to the following:	I have completed the project development and documentation. I now turn to the presentation.
Issues:	• Compiling the server code functionally • CPE Index-based mapping performs poorly
Milestones accomplished the week of 4/18/26 – 4/30/26:	• Asset pages (Vulnerabilities, Agents, Software) functional with sub-pages, filters, searching, etc. • Server compiles and is installable via .exe • CPEs map automatically to some degree • Development complete
Milestones planned this week, but not achieved with variance:	
Milestones planned for next week:	• Prepare and present
Areas/questions for discussion:	

Tasks Complete: 61 Tasks Remaining: 1 Tasks Late: 0

User's Guide

1. Server Section

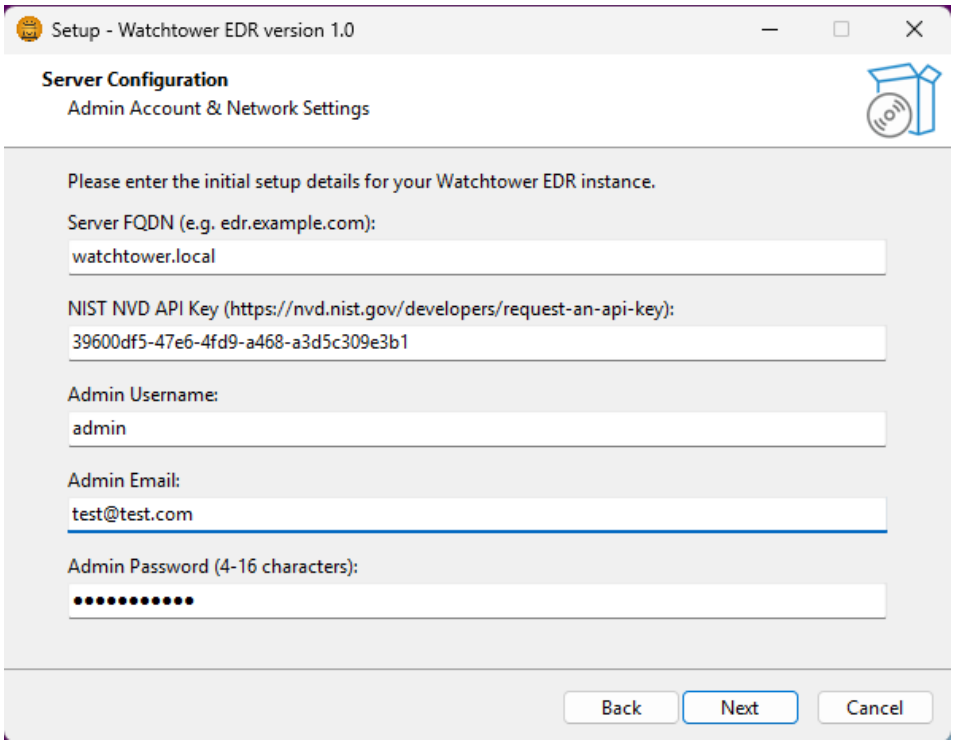
1.1 Installation

To install Watchtower EDR, run the *WatchtowerServerInstaller.exe* on the machine that will be the server.

In the installation wizard's installation location prompt, click *Next*.

On the Server Configuration screen, enter the following data:

- **Server FQDN:** The address the server will be reached at. This can be the hostname of the machine or any domain name that will resolve to the server.
- **NIST NVD API Key:** The API Key received from NIST for interfacing with their servers. Go to https://nvd.gov/developers/request-an-api-key to get a key. Ensure the key is backed up in another location for safety.
- **Admin Username:** The username of the user that will be accessing the server. This can be left as "admin".
- **Admin Email:** An email address the user can be reached at.
- **Admin Password:** Set the password for the user. Be cautious, as passwords cannot be reset currently. The password must be between 4 and 16 characters in length.



Click *Next* when all fields are completed.

At the Start Menu Folder dialogue, click *Next.*

Click *Install* on the Ready to Install screen.

Close the installer when it completes.

1.2 Configuration

System behavior is governed by the config.yaml file. Modification of database paths or NVD URLs is not recommended.

NIST Vulnerability Database Connection

A NIST API Key is required for vulnerability correlation. You can request one at NVD Developers.

- **api_key:** Your unique NVD key. Ensure you save a secondary copy of this key.
- **cpe_url / cve_url:** Default REST endpoints for NIST 2.0 data.

Server Network & UI Settings

- **fqdn:** The Fully Qualified Domain Name of the server. This must be reachable by all remote agents to ensure connectivity.
- **listen_address:** Defaults to 0.0.0.0 to listen on all available interfaces.
- **theme:** Customizable Web-UI color schemes. Options include: *blue, lightblue, green, yellow, orange, red, pink*.

Agent Communication & Security

- **offline_timer:** The threshold (in minutes) after which an agent is marked "Offline" if no heartbeat is received. Accepted values: 2 to 1440.
- **telemetry_freq:** The interval (in minutes) at which agents transmit software inventory data. Accepted values: 1 to 30.
- **enrollment_token:** A pre-shared secret used to authenticate new agents. The agent's local token must match this string to successfully enroll.
    - *Note: If this token is modified, all subsequent installation scripts must be re-downloaded from the server*.

1.3 The Dashboard

Accessing the Console

To access the management console, navigate to https://{your_server_fqdn} in a supported web browser. Log in using the administrative credentials established during the server installation process.

User Management

To view/update the username, email, or password. Select the button containing the username in the top right corner and click *Profile*. Make desired changes and click *Save Changes*. *(Password entries still have a 4–16-character requirement)*

Navigation & Metrics

The Dashboard provides an immediate overview of your security posture:

- **Persistent Navigation:** The left-hand sidebar allows for rapid switching between Agents, Inventory, Vulnerabilities, Reporting, and Settings.
- **Metric Tiles:** High-level counts for total Agents, Offline Agents, discovered Vulnerabilities, and unique Software entries. Clicking **"View All"** on any tile redirects you to the detailed page for that category.
- **Authentication Management:** To log out or view your profile, click your username in the top-right corner and select the appropriate option from the dropdown menu.

1.4 Agents Page

The Agents page is the primary interface for managing and monitoring your deployed endpoints.

Searching & Filtering

Refine the agent list using the multi-asset filter system:

- **Search Bar:** Execute text-based queries against hostnames, IP addresses, or Operating System names.
- **Category Filter:** Filter endpoints by user-defined organizational units (e.g., "Server," "Workstation").
- **Status Filter:** Toggle visibility for *Active, Offline,* or *Decommissioned* agents. By default, decommissioned agents are excluded from the view.

Agent Inventory Table

The table provides a real-time summary of all filtered endpoints:

- **Status Indicators:** Visual cues indicate connection health—**Green** (Active), **Yellow** (Offline), and **Red** (Decommissioned).
- **System Details:** Displays the reported Hostname, IPv4 Address, OS Name, and assigned Category.
- **Details Action:** Clicking the **"Details"** button opens the Agent Overlay for advanced management.

Agent Overlay

The Overlay provides deep-dive system context and administrative controls:

- **Inventory Management:** Update the agent's assigned Category or add operational notes in the Description field. Click **"Save Changes"** to persist these updates.
- **System Context:** View technical metadata including OS Version, OS Build, Binary Version, and specific OS CPE strings. This view also tracks "First Seen" and "Last Seen" timestamps.
- **Decommission Agent:** Use the red button in the bottom-left corner to initiate a remote uninstallation. This command instructs the agent to remove itself from the host system.

Add Agent Button

View 2. Agents Section/Installation for details

1.5 Software Inventory Page

The Software Master Table

This table provides a deduplicated view of software. Each row represents a unique combination of a software package and its specific version.

- **Software Name:** The name of the application as identified by the host operating system.
- **Version:** The specific build or version number of the software.
- **Vendor:** The entity responsible for the software (e.g., *Oracle Corporation*).
- **CPE:** The **Common Platform Enumeration** string. This is a standardized naming scheme used by Watchtower to correlate this software with the NIST Vulnerability Database.
- **Agent Count:** Indicates the total number of endpoints that have reported this specific software version.
- **Action:** Provides a **"Details"** button to open the Software Overlay.

Search & Filtering

Administrators can use the search bar to locate specific software packages or versions. This is particularly useful for identifying "Shadow IT" (unauthorized software) or auditing the presence of specific tools like "OpenSSL" or "Java" across the fleet.

Software Detail Overlay

When the **"Details"** button is selected, an overlay appears providing granular visibility into where that specific software exists.

- **Manually Map CPE:** Allows the user to manually bind a CPE to a software entry. This feature is best if the automatic mapping function does a poor job on a particular software.
    - Use the search features to fine-tune the results and, if an applicable CPE is returned, click *Select* to map it. If no CPE exists, the *Unbind CPE* button can be used to manually set the software to no CPE.
    - The software table will distinguish between manual and automated CPE matches to make this process easier.
- **Affected Agents List:** A comprehensive list of every agent currently running this software.

1.6 Vulnerability Management Page

This page is the analytical engine of Watchtower EDR, where software inventory meets security intelligence.

Vulnerability Correlation

Watchtower performs an automated "join" between your **Software Master Table** (via the CPE string) and the **CVE Database**.

- **CVE ID:** The unique identifier for a known vulnerability (e.g., *CVE-2024-1234*).
- **CVSS Score:** The severity rating assigned by NIST. Watchtower color-codes these to highlight **Critical (9.0-10.0)** and **High (7.0-8.9)** risks that require immediate patching.
- **Severity:** A ranking based on CVSS Score.
- **Impact Score:** Measures the amount of damage a vulnerability will cause if successful.
- **Exploit Score:** Measures how difficult a vulnerability is to trigger.
- **Vulnerability Description:** A summary of the weakness and the potential impact (e.g., *Remote Code Execution*).

Remediation Mapping

By selecting a vulnerability, the system shows exactly which software package is responsible, how severe the vulnerability is, what exactly the vulnerability is, and which agents are affected so that Administrators can then pivot to an informed and effective response.

1.7 Reporting Page

The Reporting page is used to extract data for external use, such as presentations or audits.

Features to come

1.8 Settings Page

Settings

The Settings section governs the behavior of the server and its interaction with NIST API and Watchtower Agents.

- **NVD API Management:** Update the NIST API key required for Watchtower to gather data from NIST.
- **Agent Settings:** Adjust how frequently the server expects heartbeats (offline_timer) before an agent is considered offline, and how often agents should send software data (telemetry_freq). Update the enrollment token required for agents to authenticate when attempting to enroll.
- **UI Customization:** Toggle the **Theme** settings to adjust the dashboard's color palette.

Server Management

Shut down the Watchtower EDR Server gracefully via the web GUI.

Logs

View system logs in an organized fashion including the time, message, level, and type.

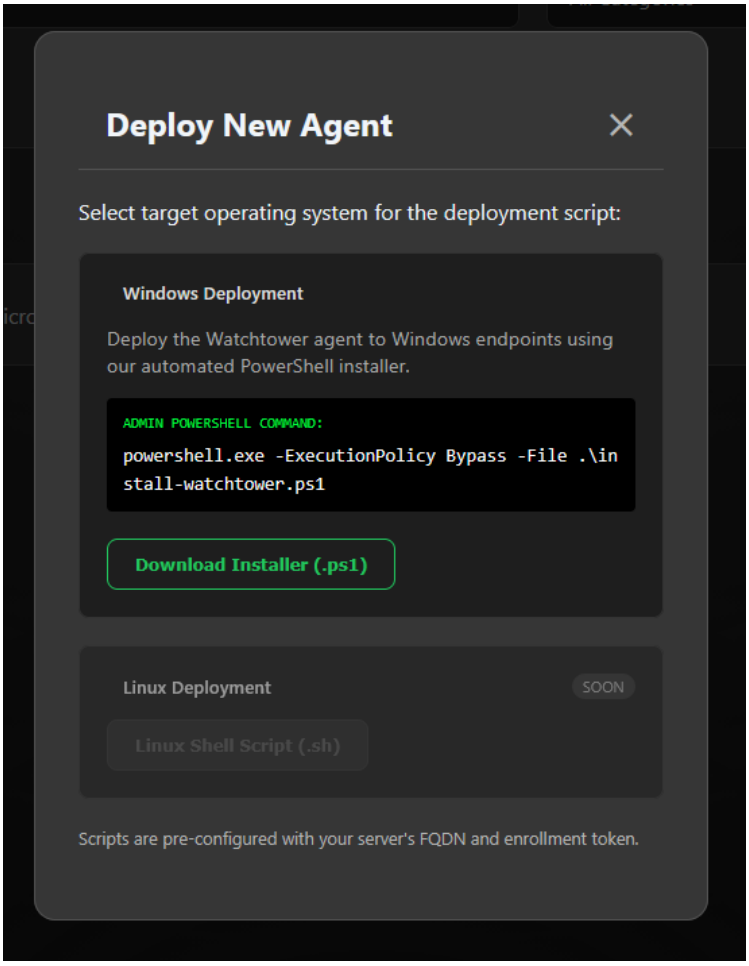
Note: Logs can be found in "{program_folder}/internal/logs/" archived log files will be compressed to save space.

2. Agent Section

2.1 Windows Installation

Step 1: Generate the Installer in the Web UI

1. Log into the Watchtower Management Console.
2. Navigate to the **Agents Page** using the sidebar.
3. Click the **"Add Agent"** button located at the top of the page. This will open the Deployment Overlay.
4. Select "**Download Installer (.ps1)**" to download the Windows installation script
   - *Note: This script is dynamically generated. It contains your server's specific FQDN and the Base64-encoded TLS certificate required for secure communication.*



Step 2: Execution via Administrative PowerShell

1. Transfer the install-watchtower.ps1 script to the target endpoint.

2. Open **PowerShell as an Administrator**.
3. Execute the script using a bypass flag to ensure the script runs regardless of local execution policies. The following script is also available to copy and paste form the Add Agent Overlay where the installer was downloaded:
    - **powershell.exe -ExecutionPolicy Bypass -File .\install-watchtower.ps1**
    - *Troubleshooting: Make sure you are running this line in the directory with the install-watchtower.ps1 script*

```
PS C:\users\fergi\Downloads> powershell.exe -ExecutionPolicy Bypass -File .\install-watchtower.ps1
--- Watchtower EDR Deployment ---
[*] Writing certificate to C:\Program Files\Watchtower\internal\server.crt...
[*] Creating config file...
[*] Downloading agent...
[+] Installation Complete.
PS C:\users\fergi\Downloads>
```

Step 3: Verification

1. Once the script reports "[+] Installation process complete!", verify the service status:
    - **Get-Service WatchtowerAgent**
2. The agent should now appear in the **Agents Page** of the Web UI with a **Green (Active)** status.

```
PS C:\users\fergi\Downloads> Get-Service WatchtowerAgent

Status   Name              DisplayName
------   ----              -----------
Running  WatchtowerAgent   Watchtower EDR Agent
```

2.2 Linux Installation

Not yet supported